DALHOUSIE UNIVERSITY

Visualizing Data

Distributions, proportions, and associations

Prof. Gabriel Spadon, PhD
Faculty of Computer Science
spadon@dal.ca

Choose a Chart That Answers the Question

- Compare distributions and explain what a summary hides.
- Show proportions with an explicit denominator.
- Examine associations before and after separating groups.
- Show change over time without hiding missing periods.
- Explain plotting choices in R or Python.

Class A covered amounts and visual design. Lab 03 combines those ideas with today's distribution, association, and trend plots.

Make Earlier Measurements Explicit

Sort a series by time before constructing lags.

$$Y_t, \quad Y_{t-1}, \quad Y_{t-2}$$

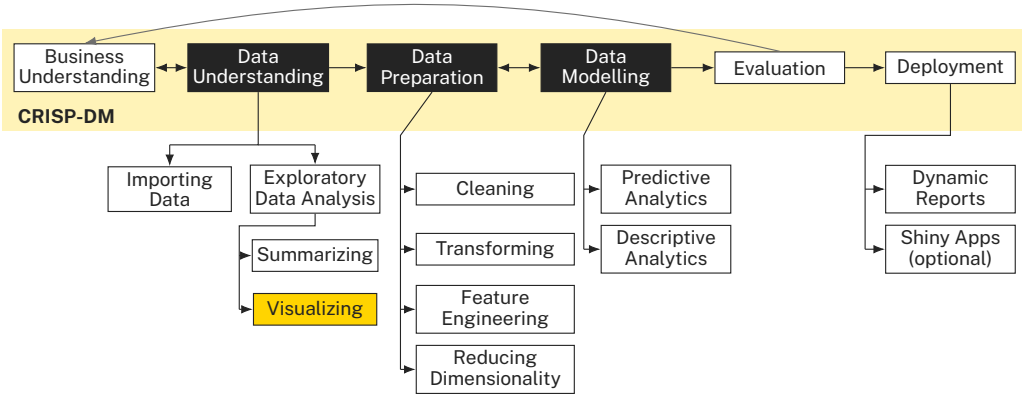
A scatter plot of Y_t against Y_{t-1} can reveal a relationship between successive measurements.

Keep separate sites or individuals separate when forming their histories.

Time	Y_t	Y_{t-1}	Y_{t-2}
1	5.1	NA	NA
2	4.8	5.1	NA
3	5.4	4.8	5.1
4	5.0	5.4	4.8

A lag of one moves a value into the next row. Initial lags are undefined; they are not zero.

Visualization in the Course Workflow



Plots check the data and its preparation; any stage can send us back.

Start with the Question

Question	Useful chart families
How much in each category?	Bars, dots, heatmaps (Class A)
How are values distributed?	Histograms, densities, boxes, violins, ECDFs, Q-Q plots
How is a whole divided?	Bars, pies, stacked bars; stacked area over time
How do two variables relate?	Scatter plots; line plots over ordered time
How do many variables relate?	Pair matrices, correlograms, parallel coordinates

The question narrows the choices; audience and sample size guide the final design.

Iris Measurements and the R Setup

The Iris data contain 150 flowers, 50 from each of three species. Four columns measure sepal and petal length and width, all in centimetres.[1]

The data ship with R and scikit-learn. They illustrate plotting, not every iris population.

Install once if missing.

```
install.packages(c("ggplot2", "GGally"))
pip install numpy pandas matplotlib seaborn
pip install scipy scikit-learn
```

R

```r
library(ggplot2)
library(GGally)
d <- iris
species <- levels(d$Species)
colours <- c("#242424", "#A88800", "#6E6E6E")
numeric_names <- names(d)[1:4]
```

The Equivalent Python Setup

DALHOUSIE UNIVERSITY

Imports

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import norm
from sklearn.datasets import load_iris
```

Data and plotting constants

```
raw = load_iris(as_frame=True)
d = raw.data.copy()
d.columns = ["Sepal.Length", "Sepal.Width",
             "Petal.Length", "Petal.Width"]
d["Species"] = pd.Categorical.from_codes(
    raw.target, raw.target_names)
species = list(raw.target_names)
colours = ["#242424", "#A88800", "#6E6E6E"]
markers = ["o", "^", "s"]
numeric_names = list(d.columns[:4])
```

Create Two Lagged Columns

R

```r
series <- data.frame(Y = c(5.1, 4.8, 5.4, 5.0))
series$lag1 <- c(NA, head(series$Y, -1))
series$lag2 <- c(NA, NA, head(series$Y, -2))
print(series)
```

Python

```python
series = pd.DataFrame({"Y": [5.1, 4.8, 5.4, 5.0]})
series["lag1"] = series["Y"].shift(1)
series["lag2"] = series["Y"].shift(2)
print(series)
```

The first lag has one undefined entry; the second has two. A lag is an earlier value, not an imputation.

Data, Mapping, Geometry, and Labels

A plotting grammar separates the data from the visual decisions.

- **Data** supply observations and variables.
- **Mappings** connect a variable to position, colour, or shape.
- **Geometry** chooses bars, points, lines, or another mark.
- **Scales, labels, and theme** make the encoding readable.

ggplot2 expresses these as layers joined by +. Python also has grammar-based libraries such as plotnine; here we use matplotlib and seaborn.

Build a Labelled Histogram

R

```r
p_grammar <- ggplot(d, aes(Sepal.Length)) +
  geom_histogram(
    breaks = seq(4.25, 8.25, 0.4),
    closed = "left") +
  labs(x = "Sepal length (cm)", y = "Count") +
  theme_minimal()
```

Python

```python
fig, ax = plt.subplots()
ax.hist(d["Sepal.Length"],
        bins=np.linspace(4.25, 8.25, 11))
ax.set(xlabel="Sepal length (cm)",
       ylabel="Count")
```

The R expression layers a geometry and labels. Python creates an axes object, draws on it, and labels that same axes.

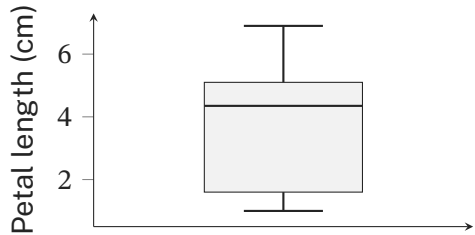
Read the Box Before Interpreting Its Shape

For petal length, the sample five-number summary is

$$1.0, \quad 1.6, \quad 4.35, \quad 5.1, \quad 6.9 \text{ cm}.$$

The box spans Q_1 to Q_3; its internal line marks the median.

Whiskers stop at observed values within $1.5\,\text{IQR}$ of the box. A point beyond them is flagged, not automatically invalid.



Compute Quartiles and Draw the Box

R

```r
x <- d$Petal.Length
print(quantile(x, c(0, .25, .5, .75, 1)))
p_box <- ggplot(d, aes(y = Petal.Length)) +
  geom_boxplot(coef = 1.5) +
  labs(y = "Petal length (cm)")
```

Python

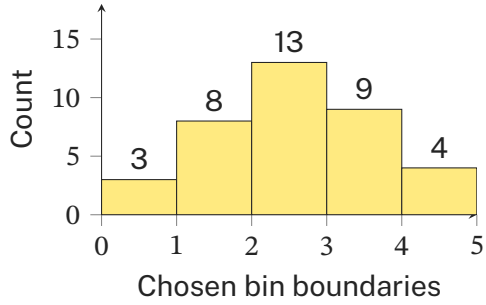
```python
x = d["Petal.Length"]
print(x.quantile([0, .25, .5, .75, 1]))
fig, ax = plt.subplots()
sns.boxplot(y=x, whis=1.5, ax=ax)
ax.set_ylabel("Petal length (cm)")
```

ggplot2 and pandas use linearly interpolated quartiles here. Base R boxplot hinges can differ on other samples. A five-number summary uses the minimum and maximum; whiskers need not reach them.

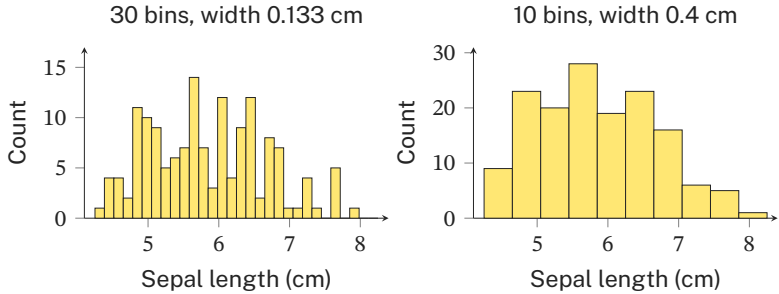
A Histogram Divides a Range into Bins

A bar chart compares named categories. A histogram counts values in chosen numeric intervals.

For equal-width bins, compare counts by bar height. With unequal widths, use count per unit width so that area represents count.



The Bin Grid Changes the Picture



Both views use the same 150 sepal lengths and range 4.25–8.25 cm. Compare 30 bins of width 0.133 cm with 10 bins of width 0.4 cm. Measurements were recorded to 0.1 cm.

Fix the Edges and Check the Counts

R

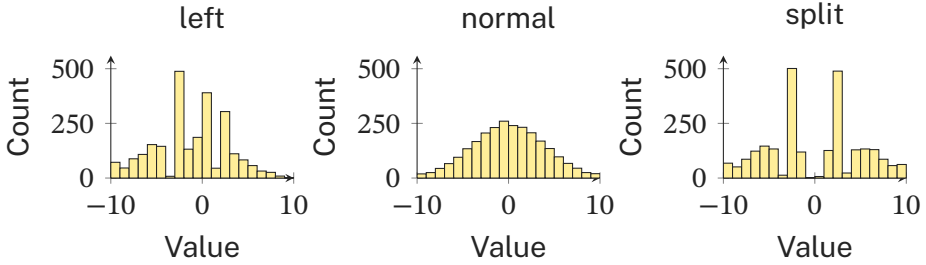
```r
edges <- seq(4.25, 8.25, by = 0.4)
bins <- cut(d$Sepal.Length, edges,
  right = FALSE, include.lowest = TRUE)
print(table(bins))
p_histogram <- ggplot(d, aes(Sepal.Length)) +
  geom_histogram(
    breaks = edges, closed = "left") +
  labs(x = "Sepal length (cm)", y = "Count")
```

Python

```python
edges = np.linspace(4.25, 8.25, 11)
counts, _ = np.histogram(
    d["Sepal.Length"], bins=edges)
print(counts)
fig, ax = plt.subplots()
ax.hist(d["Sepal.Length"], bins=edges)
ax.set(xlabel="Sepal length (cm)",
       ylabel="Count")
```

The ten counts are 9, 23, 20, 28, 19, 23, 16, 6, 5, 1. These off-grid edges avoid ties at the boundaries. Record edges and endpoint rules, not just a bin count.

Similar Summaries Hide Different Shapes



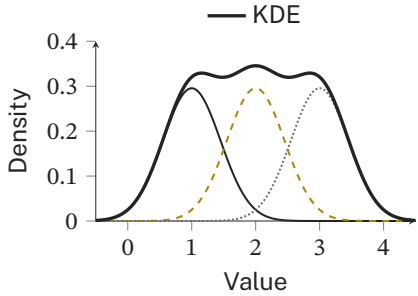
These three source datasets have nearly equal five-number summaries, approximately $(-9.77, -2.69, 0, 2.68, 9.76)$. Their equal-width histograms differ sharply. Data from Matejka and Fitzmaurice (2017), via datasauRus.

Averaging Smooth Density Contributions

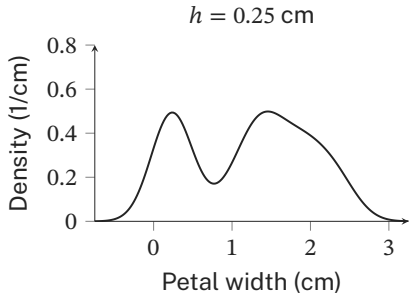
Place a bell-shaped Gaussian kernel at each observation and average them.

$$\widehat{f}_h(t) = \frac{1}{nh} \sum_{i=1}^{n} K\Big(\frac{t - x_i}{h}\Big).$$

n counts observations; x_i is one value and t is a location on the axis. The bandwidth h controls each kernel's width. The total density has area one.



Bandwidth Controls Smoothing



Here $h = 0.25$ cm for petal width. Smaller bandwidths reveal more local variation; larger ones smooth more strongly.

The vertical axis is density in cm^{-1}, not a count or a probability at one exact value.

Gaussian kernels extend below zero even though widths cannot. This boundary artefact belongs to the estimator.

Use the Same Bandwidth in Both Languages

DALHOUSIE UNIVERSITY

R

```r
h <- 0.25  # bandwidth in cm
x <- d$Petal.Width
grid <- seq(-0.75, 3.25, length.out = 201)
f <- sapply(grid, function(t) {
  mean(dnorm(t, x, h))
})
curve <- data.frame(width = grid, density = f)
p_density <- ggplot(curve,
  aes(width, density)) + geom_line() +
  labs(x = "Petal width (cm)",
       y = "Density (1/cm)")
```

Use the Same Bandwidth in Both Languages

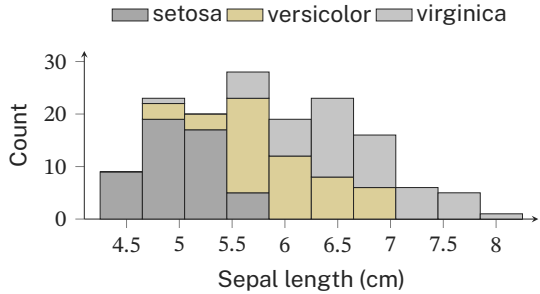
DALHOUSIE UNIVERSITY

Python

```
h = 0.25  # bandwidth in cm
x = d["Petal.Width"].to_numpy()
grid = np.linspace(-0.75, 3.25, 201)
kernels = norm.pdf(grid[:, None],
                   loc=x, scale=h)
f = kernels.mean(axis=1)
fig, ax = plt.subplots()
ax.plot(grid, f)
ax.set(xlabel="Petal width (cm)",
       ylabel="Density (1/cm)")
```

The same grid, formula, and bandwidth give matching curves. R nrd0 and SciPy Scott defaults choose different bandwidths; check the result.

Stacked Counts Emphasize the Total



Each species has 50 observations. The stack shows total counts in each interval, but only the bottom group starts at zero. Interior segment heights are harder to compare across bins.

Stack Groups in an Explicit Order

R

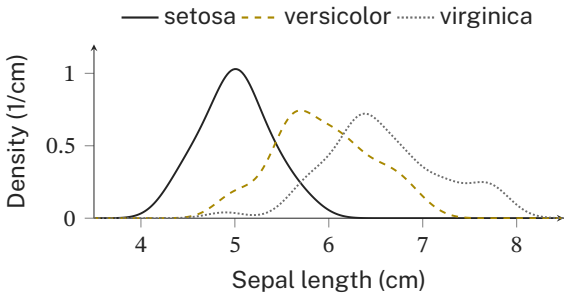
```r
fills <- c("#A7A7A7", "#DCCF99", "#C5C5C5")
styles <- c("solid", "dashed", "dotted")
p_group_hist <- ggplot(d,
  aes(Sepal.Length, fill = Species)) +
  geom_histogram(
    breaks = edges, closed = "left",
    position = position_stack(reverse = TRUE)) +
  scale_fill_manual(values = fills) +
  labs(x = "Sepal length (cm)", y = "Count")
```

Python

```python
fills = ["#A7A7A7", "#DCCF99", "#C5C5C5"]
styles = ["-", "--", ":"]
fig, ax = plt.subplots()
groups = [
    d.loc[d["Species"] == s, "Sepal.Length"]
    for s in species]
ax.hist(groups, bins=edges, stacked=True,
        color=fills, label=species)
ax.set(xlabel="Sepal length (cm)",
       ylabel="Count")
ax.legend()
```

Use the same edges for every species. To compare within-group shapes more directly, use facets, normalized densities, boxes, or violins.

Compare Curves on a Common Density Scale



Each curve has area one and uses a Gaussian bandwidth of 0.20 cm. Setosa concentrates at shorter sepal lengths; the species distributions overlap. Line styles reinforce the colour mapping.

Normalize Each Group Separately

DALHOUSIE UNIVERSITY

R

```
h_group <- 0.20  # common bandwidth, cm
p_group_density <- ggplot(d,
  aes(Sepal.Length, colour = Species,
      linetype = Species)) +
  geom_density(bw = h_group) +
  scale_colour_manual(values = colours) +
  scale_linetype_manual(values = styles) +
  labs(x = "Sepal length (cm)",
       y = "Density (1/cm)")
```

Python passes h/s because SciPy multiplies the factor by each group's SD. The bandwidth remains 0.20 cm.

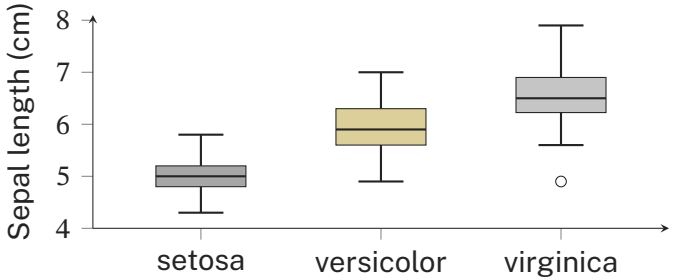
Normalize Each Group Separately

DALHOUSIE UNIVERSITY

Python

```
h_group = 0.20  # common bandwidth, cm
fig, ax = plt.subplots()
for values, s, c, line in zip(
        groups, species, colours, styles):
    bandwidth = h_group / values.std(ddof=1)
    sns.kdeplot(
        values, bw_method=bandwidth,
        common_norm=False, color=c,
        linestyle=line, label=s, ax=ax)
ax.set(xlabel="Sepal length (cm)",
       ylabel="Density (1/cm)")
ax.legend()
```

Compare Location and Spread with Boxes



The box plots use linear-interpolation quartiles. Virginica has $Q_1 = 6.225$ cm and one flagged low value at 4.9 cm. A flag prompts investigation; it is not a deletion rule.

Use One Quartile Convention

R

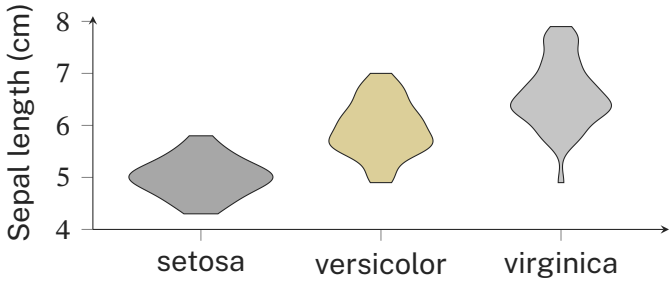
```r
p_group_box <- ggplot(d,
  aes(Species, Sepal.Length, fill = Species)) +
  geom_boxplot(coef = 1.5) +
  scale_fill_manual(values = fills) +
  labs(x = "Species", y = "Sepal length (cm)")
```

Python

```python
fig, ax = plt.subplots()
sns.boxplot(
    data=d, x="Species", y="Sepal.Length",
    hue="Species", order=species, whis=1.5,
    palette=fills, saturation=1,
    legend=False, dodge=False, ax=ax)
ax.set_ylabel("Sepal length (cm)")
```

Report the convention when exact box edges matter. Sample quartiles, Tukey hinges, whiskers, and fences describe related but distinct quantities.

A Violin Shows a Mirrored Density



The measurement axis runs vertically. The left and right sides mirror the same density, so skewness is read along the measurement axis. Bandwidth is 0.20 cm; tails are trimmed to observed ranges.

State the Violin Scaling and Tail Rule

R

```r
p_violin <- ggplot(d,
  aes(Species, Sepal.Length, fill = Species)) +
  geom_violin(bw = h_group, trim = TRUE,
              scale = "area") +
  scale_fill_manual(values = fills) +
  labs(x = "Species", y = "Sepal length (cm)")
```

Equal-area scaling applies before trimming. Visible areas can differ after trimming; the boundary estimates shape.

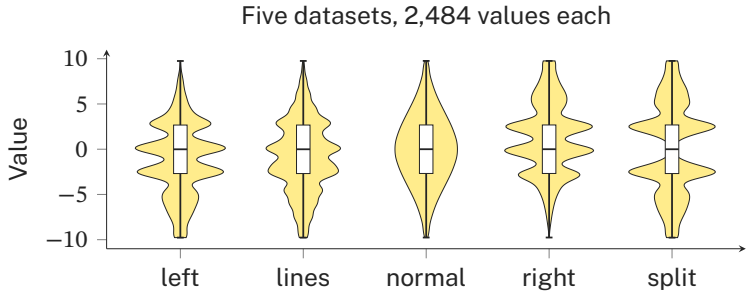
State the Violin Scaling and Tail Rule

DALHOUSIE UNIVERSITY

Python

```
fig, ax = plt.subplots()
def bandwidth(kde):
    return h_group / kde.dataset.std(ddof=1)
sns.violinplot(
    data=d, x="Species", y="Sepal.Length",
    hue="Species", order=species,
    palette=fills, saturation=1,
    legend=False, dodge=False,
    bw_method=bandwidth, cut=0,
    density_norm="area", common_norm=True,
    inner=None, ax=ax)
ax.set_ylabel("Sepal length (cm)")
```

Inspect Shape Alongside the Box



Five published datasets contain 2,484 values each. Their boxes are nearly identical, while their density shapes differ. The names identify the source datasets; each violin uses $h = 0.6$. Data from Matejka and Fitzmaurice (2017), via datasauRus.

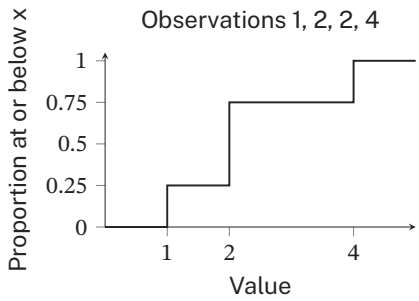
Count the Fraction at or Below a Value

The empirical cumulative distribution function equals

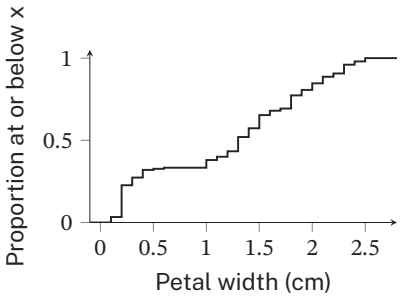
$$\widehat{F}(t) = \frac{\#\{x_i \le t\}}{n}.$$

For $1, 2, 2, 4$, the jump at 2 is $2/4$, because two observations are tied.

The ECDF is right-continuous and needs no bin width or smoothing bandwidth.



Read a Proportion or a Threshold



At 1.3 cm, the ECDF is 0.52. Thus 78 of the 150 recorded petal widths are at or below 1.3 cm.

Reading horizontally finds values at a chosen cumulative proportion.

A flat section has no recorded values in that interval. A jump counts one value or a tied group.

Plot the Empirical Cumulative Distribution

R

```r
x <- d$Petal.Width
F_hat <- ecdf(x)
print(F_hat(1.3))
p_ecdf <- ggplot(d, aes(Petal.Width)) +
  stat_ecdf() +
  labs(x = "Petal width (cm)",
       y = "Proportion at or below x")
```

Python

```python
x = d["Petal.Width"].to_numpy()
print(np.mean(x <= 1.3))
fig, ax = plt.subplots()
sns.ecdfplot(x=x, ax=ax)
ax.set(xlabel="Petal width (cm)",
       ylabel="Proportion at or below x")
```

ECDFs work for discrete and continuous variables. They describe the observed sample; population conclusions need a sampling argument.

Build the ECDF from Counts

R

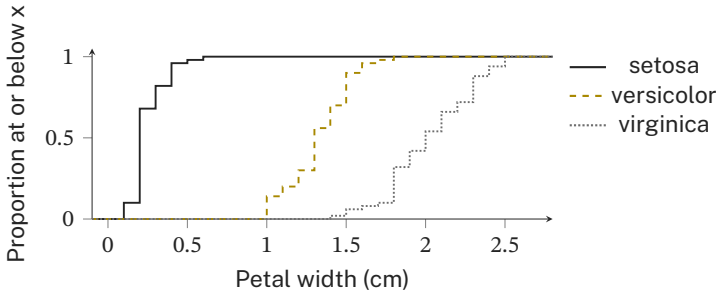
```r
values <- sort(unique(x))
counts <- tabulate(match(x, values))
proportion <- cumsum(counts) / length(x)
print(data.frame(values, counts, proportion))
stopifnot(all.equal(proportion, F_hat(values)))
```

Python

```python
values, counts = np.unique(
    x, return_counts=True)
proportion = np.cumsum(counts) / len(x)
print(pd.DataFrame({
    "values": values, "counts": counts,
    "proportion": proportion}))
direct = [(x <= v).mean() for v in values]
assert np.allclose(proportion, direct)
```

Count each distinct value, accumulate those counts, and divide by the total. A complete step plot also extends flat at zero below the minimum and flat at one above the maximum.

Compare Entire Cumulative Distributions



Setosa reaches one before versicolor begins. Versicolor is at or above virginica throughout this sample; these two curves do not cross. Curves can overlap, so retain distinct line styles.

Keep the Same Axes Across Groups

R

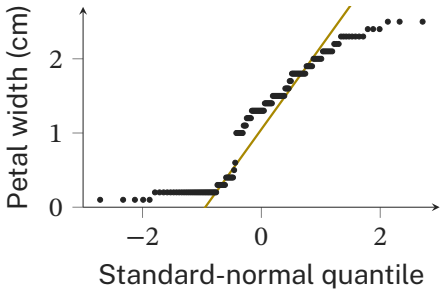
```r
p_group_ecdf <- ggplot(d,
  aes(Petal.Width, colour = Species,
      linetype = Species)) +
  stat_ecdf() +
  scale_colour_manual(values = colours) +
  scale_linetype_manual(values = styles) +
  labs(x = "Petal width (cm)",
       y = "Proportion at or below x")
```

Python

```python
fig, ax = plt.subplots()
for s, c, line in zip(
        species, colours, styles):
    values = d.loc[d["Species"] == s,
                   "Petal.Width"]
    sns.ecdfplot(values, color=c,
                 linestyle=line, label=s,
                 ax=ax)
ax.set(xlabel="Petal width (cm)",
       ylabel="Proportion at or below x")
ax.legend()
```

A vertical separation compares cumulative proportions at one value. A horizontal separation compares sample quantiles at a chosen proportion.

A Q-Q Plot Compares Quantiles



A normal Q-Q plot compares ordered observations with quantiles of a normal reference having mean zero and SD one.

Points near a straight line are consistent with a normal shape after shifting and rescaling. Sampling variation prevents exact alignment.

Petal width departs from this reference. The pattern alone does not identify a unique mixture or cause.

Use the Same Q-Q Positions and Reference

R

```r
n <- length(x)
q <- qnorm((seq_len(n) - 0.5) / n)
quartiles <- quantile(x, c(.25, .75))
slope <- diff(quartiles) /
  diff(qnorm(c(.25, .75)))
intercept <- quartiles[1] - slope * qnorm(.25)
qq_data <- data.frame(q = q, value = sort(x))
p_qq <- ggplot(qq_data, aes(q, value)) +
  geom_point() +
  geom_abline(intercept = intercept,
              slope = slope) +
  labs(x = "Standard-normal quantile",
       y = "Petal width (cm)")
```

Python

```python
n = len(x)
prob = (np.arange(1, n + 1) - 0.5) / n
q = norm.ppf(prob)
quartiles = np.quantile(x, [.25, .75])
normal_iqr = np.diff(norm.ppf([.25, .75]))[0]
slope = np.diff(quartiles)[0] / normal_iqr
intercept = quartiles[0] - slope * norm.ppf(.25)
fig, ax = plt.subplots()
ax.scatter(q, np.sort(x))
ax.plot(q, intercept + slope * q)
ax.set(xlabel="Standard-normal quantile",
       ylabel="Petal width (cm)")
```

The line passes through the sample first and third quartiles.

Define the Whole Before Drawing Its Parts

R

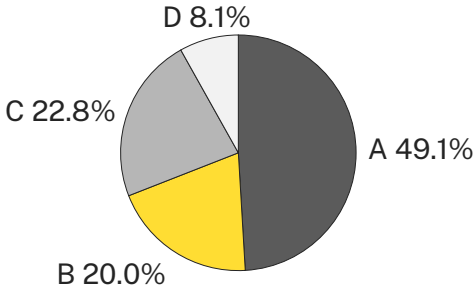
```r
parts <- data.frame(
  Party = factor(LETTERS[1:4],
                 levels = LETTERS[1:4]),
  Count = c(157, 64, 73, 26))
parts$Prop <- parts$Count / sum(parts$Count)
print(parts)
p_proportions <- ggplot(parts,
  aes(Party, Prop)) +
  geom_col() + labs(y = "Proportion")
```

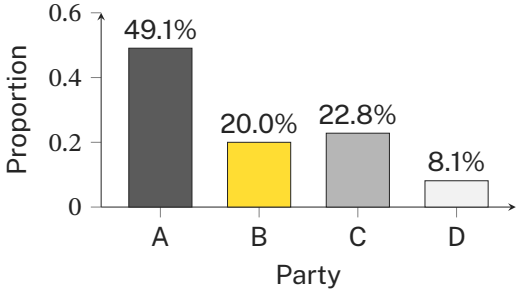
Python

```python
parts = pd.DataFrame({
    "Party": list("ABCD"),
    "Count": [157, 64, 73, 26]})
total = parts["Count"].sum()
parts["Prop"] = parts["Count"] / total
print(parts)
ax = parts.plot.bar(
    x="Party", y="Prop", legend=False)
ax.set_ylabel("Proportion")
```

The seat counts total 320. A proportion divides a count by that total. Changing the denominator changes the question.

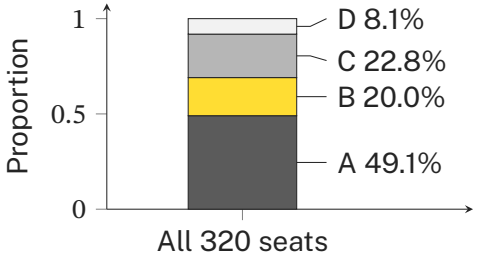
Angles and Lengths Show the Same Shares





A = 49.0625%, B = 20%, C = 22.8125%, D = 8.125%. Separate bars make the small B/C difference easier to compare on a common baseline.

A Stack Emphasizes Composition



The four segment heights sum to one. The two interior segments do not share a baseline.

A normalized stack compares composition, not absolute totals. Two groups can have identical shares and very different sizes.

Create One Stacked Bar

R

```r
p_stacked <- ggplot(parts,
  aes(x = "All", y = Prop, fill = Party)) +
  geom_col(
    position = position_stack(reverse = TRUE)) +
  labs(x = NULL, y = "Proportion")
p_pie <- p_stacked + coord_polar(theta = "y") +
  theme_void()
```

Python

```python
one_row = parts.set_index("Party")[["Prop"]].T
stacked_ax = one_row.plot.bar(stacked=True)
stacked_ax.set(xlabel="", ylabel="Proportion")
fig, ax = plt.subplots()
ax.pie(parts["Count"], labels=parts["Party"],
       startangle=90, counterclock=False)
```

Python first turns the four proportions into one row with four columns. Pie orientation can differ between libraries; category shares remain the same.

Choose the Encoding for the Comparison



A pie shows parts of one whole.
Similar angles can be hard to compare.



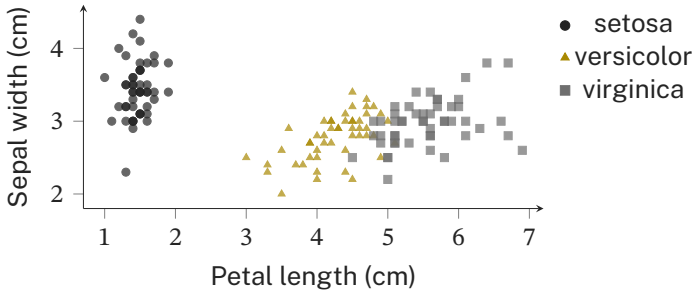
A stack emphasizes composition.
Interior segments start at different heights.



Bars use a shared baseline.
Compare their lengths directly.

No chart is the best choice for every task. Include labels or exact values when readers need precise comparisons.

Inspect Two Numeric Variables



Each point is one flower. Position encodes two measurements; colour and marker shape identify species. The pooled relationship can differ from the relationship within each species.

Map Species to Colour and Shape

R

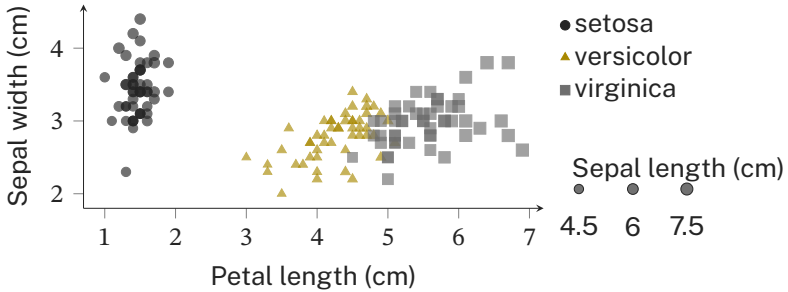
```r
p_scatter <- ggplot(d,
  aes(Petal.Length, Sepal.Width,
      colour = Species, shape = Species)) +
  geom_point(alpha = .7) +
  scale_colour_manual(values = colours) +
  scale_shape_manual(values = c(16, 17, 15)) +
  labs(x = "Petal length (cm)",
       y = "Sepal width (cm)")
```

Python

```python
fig, ax = plt.subplots()
sns.scatterplot(
    data=d, x="Petal.Length", y="Sepal.Width",
    hue="Species", style="Species",
    hue_order=species, style_order=species,
    markers=markers, palette=colours,
    alpha=.7, ax=ax)
ax.set(xlabel="Petal length (cm)",
       ylabel="Sepal width (cm)")
```

Redundant colour and shape help readers distinguish groups. Transparency can reveal overlap, but it does not turn overplotted points into an exact density estimate of the data.

A Fourth Variable Can Use Marker Area



Within one marker shape, area is proportional to sepal length. Different shapes have different area constants; compare sizes within a species. The size key uses circles.

Keep Area Proportional to the Measurement

R

```r
p_size <- p_scatter + aes(size = Sepal.Length) +
  scale_size_area(
    max_size = 5,
    limits = c(0, max(d$Sepal.Length)),
    breaks = c(4.5, 6, 7.5),
    name = "Sepal length (cm)")
```

Python

```python
fig, ax = plt.subplots()
maximum = d["Sepal.Length"].max()
sns.scatterplot(
    data=d, x="Petal.Length", y="Sepal.Width",
    hue="Species", style="Species",
    hue_order=species, style_order=species,
    markers=markers, palette=colours,
    size="Sepal.Length", size_norm=(0, maximum),
    sizes=(0, 120), alpha=.7, ax=ax)
ax.set(xlabel="Petal length (cm)",
       ylabel="Sepal width (cm)")
```

Within a fixed shape, a value twice as large receives twice the area. R and Python use different physical size units. Cross-shape area comparisons need separate calibration.

Pooling Groups Can Change the Relationship

Pearson's r summarizes linear association, from -1 to $+1$. It has no unit and does not establish causation.

Group	n	r(petal length, sepal width)
All flowers	150	-0.428
Setosa	50	$+0.178$
Versicolor	50	$+0.561$
Virginica	50	$+0.401$

All three within-species values are positive, while the pooled value is negative. Between-species differences and within-species variation answer different questions.

Compute Pooled and Grouped Correlations

R

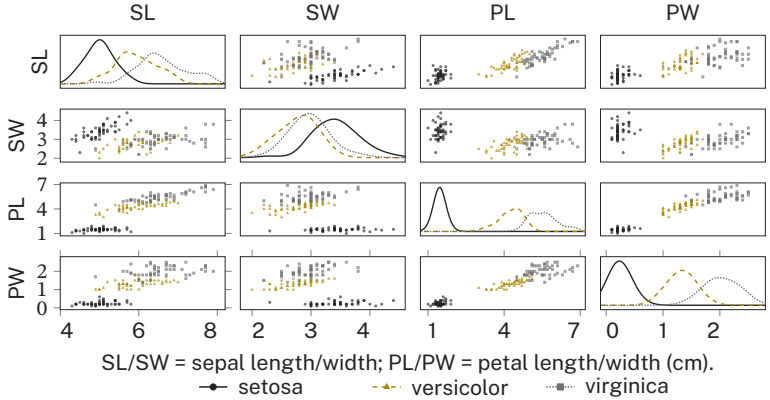
```r
print(cor(d$Petal.Length, d$Sepal.Width))
by_species <- by(d, d$Species, function(g)
  cor(g$Petal.Length, g$Sepal.Width))
print(by_species)
correlations <- cor(d[numeric_names])
print(round(correlations, 3))
```

Python

```python
print(d["Petal.Length"].corr(d["Sepal.Width"]))
by_species = {}
for s, g in d.groupby("Species", observed=True):
    by_species[s] = g["Petal.Length"].corr(
        g["Sepal.Width"])
print(by_species)
correlations = d[numeric_names].corr()
print(correlations.round(3))
```

Use the same eligible records and report their count. A large absolute correlation does not rule out outliers, nonlinear structure, or important group differences.

Use a Matrix to Inspect Variable Pairs



Six pairs, each drawn twice with axes exchanged; the diagonal shows density.

Select the Same Four Numeric Variables

R

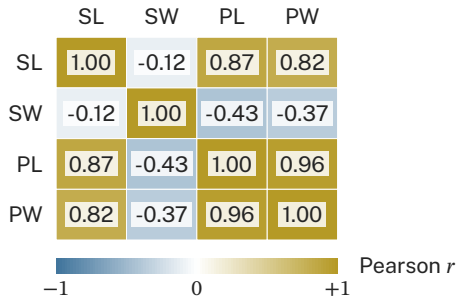
```r
p_matrix <- ggpairs(d, columns = 1:4,
  mapping = aes(colour = Species,
                shape = Species),
  upper = list(continuous = "points"),
  lower = list(continuous = "points"),
  diag = list(continuous = wrap(
    "densityDiag", bw = h_group))) +
  scale_colour_manual(values = colours) +
  scale_shape_manual(values = c(16, 17, 15))
```

Python

```python
matrix = sns.pairplot(
    d, vars=numeric_names, hue="Species",
    palette=colours, markers=markers,
    diag_kind="kde",
    diag_kws={"bw_method": bandwidth,
              "common_norm": False})
```

Both matrices use the four centimetre-valued measurements. Species defines groups rather than a fifth numeric axis. A matrix is an overview; enlarge a selected pair before making a detailed claim.

A Correlogram Maps Linear Relationships



SL = sepal length; SW = sepal width; PL = petal length; PW = petal width.

The colour scale runs from -1 through zero to $+1$. Printed values retain two decimals, so $r = 0.963$ is shown as 0.96, not as a perfect correlation.

The diagonal is one by construction. The matrix is symmetric. Return to scatter plots to inspect shape and groups.

Use the Numbers and Colour Scale Together

R

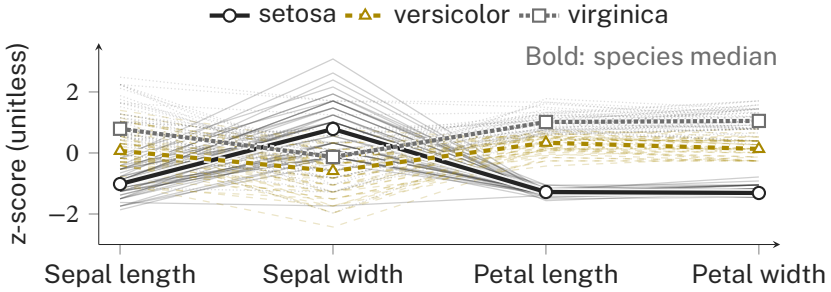
```r
cells <- as.data.frame(as.table(correlations))
p_correlogram <- ggplot(cells,
  aes(Var1, Var2, fill = Freq)) + geom_tile() +
  geom_text(aes(label = sprintf("%.2f", Freq))) +
  scale_fill_gradient2(
    low = "#506C85", mid = "white",
    high = "#A88800", limits = c(-1, 1)) +
  coord_equal() +
  labs(x = NULL, y = NULL, fill = "r")
```

Python

```python
fig, ax = plt.subplots()
palette = sns.diverging_palette(
    240, 60, as_cmap=True)
sns.heatmap(
    correlations, annot=True, fmt=".2f",
    vmin=-1, vmax=1, center=0, square=True,
    cmap=palette, ax=ax)
```

Fix the colour limits at -1 and +1 when comparing correlation matrices. A shared numerical scale is needed even when the libraries use different palettes.

Parallel Coordinates Show One Row



Each line is one flower. Values are centered and divided by the sample SD, so the vertical axis is a unitless z-score. Axis order changes which crossings are visible.

Compare Standardized Column Profiles

R

```r
z <- scale(d[numeric_names])
n <- nrow(d)
variable <- factor(numeric_names,
                   levels = numeric_names)
long <- data.frame(Row = rep(seq_len(n), 4),
  Variable = rep(variable, each = n),
  Z = as.vector(z), Species = rep(d$Species, 4))
p_parallel <- ggplot(long, aes(Variable, Z,
  group = Row, colour = Species,
  linetype = Species)) + geom_line(alpha = .35) +
  scale_colour_manual(values = colours) +
  scale_linetype_manual(values = styles) +
  labs(x = NULL, y = "z-score (unitless)")
```

Python

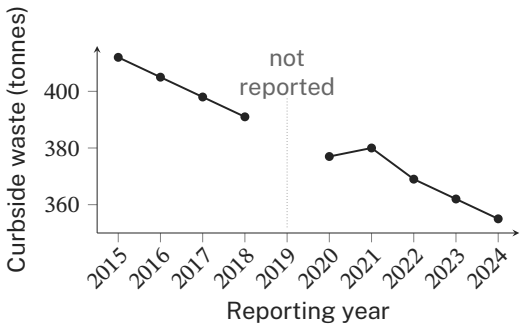
```python
z = d[numeric_names]
z = (z - z.mean()) / z.std(ddof=1)
fig, ax = plt.subplots()
for s, c, line in zip(species, colours, styles):
    rows = z.loc[d["Species"] == s].to_numpy()
    ax.plot(range(4), rows.T, color=c,
            linestyle=line, alpha=.35)
    ax.plot([], [], color=c, linestyle=line,
            label=s)
ax.set_xticks(range(4), numeric_names)
ax.set_ylabel("z-score (unitless)")
ax.legend()
```

Both examples use the sample SD, with divisor $n - 1$.

A Line Joins Ordered Observations

Place the ordered variable, such as year, on the horizontal axis. A line then shows the change between adjacent periods.

The 2019 value is missing. The line breaks there rather than dropping to zero or bridging the gap.



Keep a Missing Period Visible

R

```r
annual <- data.frame(Year = 2015:2024,
  Tonnes = c(412, 405, 398, 391, NA,
             377, 380, 369, 362, 355))
p_line <- ggplot(annual, aes(Year, Tonnes)) +
  geom_line() + geom_point() +
  scale_x_continuous(breaks = 2015:2024) +
  labs(x = "Reporting year",
       y = "Curbside waste (tonnes)")
```

Python

```python
annual = pd.DataFrame({
    "Year": range(2015, 2025),
    "Tonnes": [412, 405, 398, 391, np.nan,
               377, 380, 369, 362, 355]})
fig, ax = plt.subplots()
ax.plot(annual["Year"], annual["Tonnes"],
        marker="o", color="black")
ax.set_xticks(range(2015, 2025))
ax.set(xlabel="Reporting year",
       ylabel="Curbside waste (tonnes)")
```

Both libraries break a line at `NA` or `NaN`. Dropping the missing row first would silently join 2018 to 2020. Keep the year and state why it is missing.

Check the Claim, Encoding, and Audience

1. What comparison should the reader make?
2. Do the variables, units, grouping, and denominator support it?
3. Could binning, smoothing, ordering, or an axis limit change the conclusion?
4. Can a reader identify the groups and reproduce the displayed values?

A chart chooser offers candidates. Perception and the analysis question determine whether a candidate works.

Build and Defend a Small Visual Argument

1. Draw two histograms of one variable using documented edges.
2. Add an ECDF or a box plot and explain what each view reveals.
3. Inspect a numeric pair, then distinguish meaningful groups.
4. Write a caption with units, eligible records, and one limitation.

Work in R or Python.

Evaluate a Chart You Encounter

Choose one public chart from a news article, a research paper, or a government statistical agency.

- Identify the question, source, units, and denominator.
- Explain one design choice that supports or weakens the claim.
- Sketch an alternative without inventing missing data.

Credit the chart and source; distinguish a design critique from a claim that its underlying data are wrong.

Sources Cited in This Class

[1] R. A. Fisher. “The use of multiple measurements in taxonomic problems”. In: *Annals of Eugenics* 7.2 (1936), pp. 179–188.

[2] J. Matejka and G. Fitzmaurice. “Same stats, different graphs: generating datasets with varied appearance and identical statistics through simulated annealing”. In: *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 2017, pp. 1290–1294.